

Project Planning

Phase 4 — Project Planning

This phase describes how the work was organised: what had to be done, in what order, who is responsible, what could go wrong, and how success is measured. Items marked *to be filled* need real information from the team.

1. Project Scope

In scope

- User registration, login and logout with JWT authentication.
- Home, Party and Jewelry budget planners powered by Gemini.
- Optional outfit image upload for the jewelry planner.
- Shopping links for every recommended item.
- Dashboard with recent recommendations and a full history page.
- Documentation, test plan and demo for all eight phases.

Out of scope (future work)

- Permanent database, payments, live price tracking, mobile app, email features, shared plans.

2. Work Breakdown

Stage A — Understanding (Phases 1 and 2)

- A1. Discuss problem and shortlist ideas.
- A2. Choose the three planners.
- A3. Write functional and non-functional requirements.
- A4. Select the technology stack.

Stage B — Design (Phase 3)

- B1. Draw the architecture and request flow.
- B2. Define routes and data models.
- B3. Define the AI prompt format and JSON result structure.
- B4. Plan the pages and navigation.

Stage C — Development (Phase 5)

- C1. Set up the project, virtual environment and requirements.txt.
- C2. Write the data models.
- C3. Build registration, login, logout and token blacklist.
- C4. Connect to Gemini and write the JSON extraction helper.
- C5. Build the Home planner.
- C6. Build the Party planner.
- C7. Build the Jewelry planner with image upload.

- C8. Add shopping link generation.
- C9. Build the dashboard and history pages.
- C10. Build HTML templates and CSS.

Stage D — Testing (Phase 6)

- D1. Write test cases.
- D2. Run manual tests for login, planners, links and history.
- D3. Record results and fix bugs.

Stage E — Documentation and Demo (Phases 7 and 8)

- E1. Write the project documentation.
- E2. Prepare the demo script and screenshots.
- E3. Rehearse and present.

3. Status Summary

- Ideation, requirements and design: done.
- Development of all listed features: done in the submitted code.
- Documentation: done.
- Test cases: written; running them and recording results is pending.
- Demo: script prepared; screenshots and recording are to be added by the team.

4. Team Roles

Responsibilities below should be confirmed by the team. Suggested split based on the work stages:

- **Nandhini.M (Team Leader)** — coordination, final integration and review. Assigned tasks: *to be filled*.
- **Amrin.J** — assigned tasks: *to be filled*.
- **Pooja Shri.R** — assigned tasks: *to be filled*.
- **Yakshitha.M** — assigned tasks: *to be filled*.

Suggested areas to distribute: authentication and models; AI integration and prompts; front-end templates and styling; testing, documentation and demo.

5. Milestones

- Milestone 1 — Idea and requirements approved. Date: *add date*.
- Milestone 2 — Design complete. Date: *add date*.
- Milestone 3 — Authentication working. Date: *add date*.
- Milestone 4 — All three planners working. Date: *add date*.
- Milestone 5 — Testing complete. Date: *add date*.
- Milestone 6 — Documentation and demo ready. Date: *add date*.

6. Development Approach

The team worked in small, testable steps: build one feature, run it in the browser, fix it, then move on. The order was chosen so each step depends only on earlier ones: models first, then authentication (because every planner needs login), then the AI layer, then the three planners, then history, and finally styling.

7. Tools Used

- Python 3 with a virtual environment.
- VS Code (or any editor) for coding.
- Git and GitHub for version control and sharing: <https://github.com/nandhinim25ds026-alt/PocketSmart-AI>
- Browser developer tools for checking requests and responses.
- Google AI Studio to create the Gemini API key.
- FastAPI's automatic documentation page (/docs) for trying routes.

8. Risk Management

Risk: Gemini returns text that is not valid JSON.

Impact: the planner fails. Mitigation: the prompt demands JSON only, code strips markdown fences, and a fallback extracts the first JSON block.

Risk: The AI suggests a plan that exceeds the budget.

Impact: misleading result. Mitigation: prompt instructs the model to stay within budget; testing should compare totals with the entered budget.

Risk: API key is leaked on GitHub.

Impact: misuse and cost. Mitigation: `.env` stays out of Git; only `.env.example` is committed; regenerate the key if exposed.

Risk: Data is lost when the server restarts.

Impact: users must register again. Mitigation: documented limitation; database planned.

Risk: The Gemini SDK or model is retired or changed.

Impact: AI calls stop working. Mitigation: move to the current SDK and model; keep model name in one place.

Risk: Shopping sites change their search URLs.

Impact: links stop working. Mitigation: platform URLs are kept in dictionaries and can be updated quickly.

Risk: API quota or rate limits during the demo.

Impact: no results shown live. Mitigation: test the day before, keep sample screenshots or a recording as backup.

Risk: Team schedule conflicts.

Impact: delays. Mitigation: small tasks, regular check-ins, shared repository.

9. Quality Plan

- Every route must be tried manually at least once.
- Every planner must be tried with a normal, a low and a very high budget.
- Code is kept in separate modules with meaningful names.
- Secrets are checked before each push to GitHub.

10. Communication Plan

- Quick updates in the team's chat group.
- A short review of progress after each milestone.
- All code changes go through the shared GitHub repository so everyone has the latest version.

11. Success Measures

- All 27 functional requirements from Phase 2 are met.
- All test cases in Phase 6 are run and results recorded.
- Live demo completes all steps in the demo script without errors.
- Documentation is complete and matches the code.

12. Future Roadmap

Short term

- Replace in-memory storage with a database such as SQLite or PostgreSQL.
- Move to the current Gemini SDK and a supported model.
- Restrict CORS to trusted origins and make SECRET_KEY mandatory.
- Improve error messages when the AI fails.

Medium term

- Automated tests with pytest.
- Rate limiting and stronger authentication (email verification, password reset).
- Save, edit and delete plans; export a plan as PDF.

Long term

- More planners (wedding, travel, gadgets).
- Live price comparison.
- Deployment with Docker and a hosting service.
- Regional language support.